

GOOGLE CLOUD PROFESSIONAL CLOUD ARCHITECT

Section 3.6 — Implementing Resources Through Infrastructure as Code

GCP Associate Cloud Engineer

Study Notes Study Notes April 2026

Exam Relevance

This topic is part of **Section 3: Deploying and implementing a cloud solution (~25 % of the exam)**. You must understand infrastructure as code (IaC) tooling including Cloud Foundation Toolkit, Config Connector, Terraform, and Helm.

1. Infrastructure as Code (IaC) Overview

What Is IaC?

Infrastructure as Code means defining and managing cloud resources using **declarative configuration files** rather than manual Console operations. Benefits:

- **Repeatability** — Deploy identical environments (dev, staging, prod)
 - **Version control** — Track changes in Git
 - **Automation** — CI/CD pipelines for infrastructure
 - **Documentation** — Configuration files serve as documentation
 - **Consistency** — Eliminate configuration drift
 - **Collaboration** — Code review for infrastructure changes
-

2. Terraform

What Is Terraform?

- **Open-source IaC tool** by HashiCorp
- Most widely used IaC tool for Google Cloud
- Declarative language (HCL — HashiCorp Configuration Language)
- Supports multiple cloud providers (multi-cloud)
- Manages resource lifecycle (create, update, delete)

Core Concepts

Concept	Description
Provider	Plugin for a cloud platform (e.g., <code>google</code> , <code>aws</code>)
Resource	A piece of infrastructure to manage (e.g., VM, VPC)
Data Source	Read-only information from existing resources
Variable	Input parameters for configurations
Output	Values returned after applying
State	Record of managed resources (stored in <code>terraform.tfstate</code>)
Module	Reusable collection of resources
Plan	Preview of changes before applying

Basic Terraform Workflow

```
terraform init → terraform plan → terraform apply → terraform destroy
|               |               |               |
|               |               |               └─ Delete all resources
|               |               └─ Create/update resources
|               └─ Preview changes (dry run)
└─ Download providers and initialize backend
```

Terraform Configuration Example

```
# main.tf

# Configure the Google provider
terraform {
  required_providers {
    google = {
      source  = "hashicorp/google"
      version = "~> 5.0"
    }
  }
  backend "gcs" {
    bucket = "my-terraform-state"
    prefix = "terraform/state"
  }
}

provider "google" {
  project = var.project_id
  region  = var.region
}

# Create a VPC
resource "google_compute_network" "vpc" {
  name                  = "my-vpc"
  auto_create_subnetworks = false
}

# Create a subnet
resource "google_compute_subnetwork" "subnet" {
  name            = "my-subnet"
  ip_cidr_range  = "10.0.1.0/24"
  region         = var.region
  network        = google_compute_network.vpc.id

  private_ip_google_access = true
}

# Create a VM instance
resource "google_compute_instance" "vm" {
  name         = "my-vm"
  machine_type = "e2-standard-4"
  zone        = "${var.region}-a"

  boot_disk {
    initialize_params {
      image = "debian-cloud/debian-12"
      size  = 50
      type  = "pd-ssd"
    }
  }
}
```

```
    }  
  }  
  
  network_interface {  
    subnetwork = google_compute_subnetwork.subnet.id  
    # No external IP (access_config block omitted)  
  }  
  
  tags = ["web-server"]  
  
  metadata = {  
    enable-oslogin = "TRUE"  
  }  
  
  service_account {  
    email  = google_service_account.vm_sa.email  
    scopes = ["cloud-platform"]  
  }  
}  
  
# Create a service account  
resource "google_service_account" "vm_sa" {  
  account_id   = "vm-service-account"  
  display_name = "VM Service Account"  
}  
  
# Create a firewall rule  
resource "google_compute_firewall" "allow_http" {  
  name     = "allow-http"  
  network = google_compute_network.vpc.name  
  
  allow {  
    protocol = "tcp"  
    ports    = ["80", "443"]  
  }  
  
  source_ranges = ["0.0.0.0/0"]  
  target_tags   = ["web-server"]  
}
```

```
# variables.tf
variable "project_id" {
  description = "GCP project ID"
  type        = string
}

variable "region" {
  description = "GCP region"
  type        = string
  default     = "us-central1"
}
```

```
# outputs.tf
output "vm_internal_ip" {
  value = google_compute_instance.vm.network_interface[0].network_ip
}

output "vpc_id" {
  value = google_compute_network.vpc.id
}
```

Terraform Commands

```
# Initialize (download providers, configure backend)
terraform init

# Format configuration files
terraform fmt

# Validate configuration
terraform validate

# Plan changes (dry run)
terraform plan

# Apply changes
terraform apply

# Apply without confirmation prompt
terraform apply -auto-approve

# Destroy all resources
terraform destroy

# Show current state
terraform show

# List resources in state
terraform state list

# Import an existing resource into state
terraform import google_compute_instance.vm projects/PROJECT/zones/ZONE/instances/NAME

# Remove a resource from state (without deleting it)
terraform state rm google_compute_instance.vm
```

Terraform State Management

Backend	Description	Use Case
Local	<code>terraform.tfstate</code> on local disk	Development only
GCS	State stored in Cloud Storage bucket	Team collaboration, production
Remote (Terraform Cloud)	HashiCorp's managed backend	Enterprise management

Best practices for state: - **Never store state locally in production** — Use a remote backend (GCS) - **Enable state locking** — Prevents concurrent modifications - **Enable versioning** on the GCS bucket — Recover from bad applies - **Never edit state manually** — Use `terraform state` commands

3. Cloud Foundation Toolkit (CFT)

What Is CFT?

- Collection of **Terraform modules** built by Google
- Best-practice reference templates for common GCP architectures
- Opinionated, production-ready configurations

Key Modules

Module	Description
<code>terraform-google-project-factory</code>	Create and configure projects
<code>terraform-google-network</code>	Create VPCs, subnets, firewall rules
<code>terraform-google-kubernetes-engine</code>	Create GKE clusters
<code>terraform-google-sql-db</code>	Create Cloud SQL instances
<code>terraform-google-iam</code>	Manage IAM bindings
<code>terraform-google-log-export</code>	Configure log exports

Example: Using CFT Network Module

```
module "vpc" {  
  source = "terraform-google-modules/network/google"  
  version = "~> 9.0"  
  
  project_id = var.project_id  
  network_name = "my-vpc"  
  routing_mode = "REGIONAL"  
  
  subnets = [  
    {  
      subnet_name      = "web-subnet"  
      subnet_ip        = "10.0.1.0/24"  
      subnet_region    = "us-central1"  
      subnet_private_access = true  
    },  
    {  
      subnet_name      = "db-subnet"  
      subnet_ip        = "10.0.2.0/24"  
      subnet_region    = "us-central1"  
      subnet_private_access = true  
    }  
  ]  
  
  firewall_rules = [  
    {  
      name      = "allow-http"  
      direction = "INGRESS"  
      allow = [{  
        protocol = "tcp"  
        ports    = ["80", "443"]  
      }]  
      source_ranges = ["0.0.0.0/0"]  
      target_tags   = ["web-server"]  
    }  
  ]  
}
```

When to Use CFT

- Deploying standard GCP architectures
- Ensuring best practices are followed
- Reducing boilerplate Terraform code
- Accelerating project setup

4. Config Connector

What Is Config Connector?

- A **Kubernetes add-on** that manages GCP resources using Kubernetes resource definitions (YAML/kubectl)
- Treats GCP resources as **Kubernetes custom resources**
- Enables GitOps workflows for infrastructure

How It Works

```
Developer → kubectl apply -f gcs-bucket.yaml → Config Connector → GCP API → Cloud Storage
```

Example: Creating a Cloud Storage Bucket

```
# bucket.yaml
apiVersion: storage.cnrm.cloud.google.com/v1beta1
kind: StorageBucket
metadata:
  name: my-config-connector-bucket
  namespace: config-connector
  annotations:
    cnrm.cloud.google.com/project-id: "my-project-id"
spec:
  location: US
  storageClass: STANDARD
  uniformBucketLevelAccess: true
  versioning:
    enabled: true
```

```
# Apply the resource
kubectl apply -f bucket.yaml

# Check status
kubectl get storagebucket my-config-connector-bucket -n config-connector

# Delete the resource (also deletes from GCP)
kubectl delete -f bucket.yaml
```

Example: Creating a Compute Instance

```
apiVersion: compute.cnrm.cloud.google.com/v1beta1
kind: ComputeInstance
metadata:
  name: my-vm
  namespace: config-connector
spec:
  machineType: e2-standard-4
  zone: us-central1-a
  bootDisk:
    initializeParams:
      sourceImageRef:
        external: debian-cloud/debian-12
  networkInterface:
    - networkRef:
        name: my-vpc
      subnetworkRef:
        name: my-subnet
```

When to Use Config Connector

- Teams already using Kubernetes and kubectl
- Want to manage GCP resources alongside K8s resources
- GitOps workflows (ArgoCD, Flux)
- Prefer YAML over HCL (Terraform)

Config Connector vs. Terraform

Feature	Config Connector	Terraform
Language	YAML (K8s manifests)	HCL
Requires	GKE cluster	Terraform CLI
Reconciliation	Continuous (controller loop)	On-demand (apply/plan)
State	Kubernetes API (etcd)	State file (local/remote)
Multi-cloud	GCP only	Multi-cloud
Learning curve	K8s knowledge needed	Terraform knowledge needed
Best for	K8s-native teams	General IaC

5. Helm

What Is Helm?

- **Package manager for Kubernetes** (like apt/brew for K8s)
- Packages Kubernetes manifests into **charts** (reusable bundles)
- Manages releases (install, upgrade, rollback)
- Supports templating for configurable deployments

Core Concepts

Concept	Description
Chart	Package of K8s resource templates
Release	An instance of a chart deployed to a cluster
Repository	Collection of charts (like a package registry)
Values	Configuration parameters for a chart
Template	K8s manifest with Go template variables

Helm Commands

```
# Install Helm
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash

# Add a chart repository
helm repo add stable https://charts.helm.sh/stable
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update

# Search for charts
helm search repo nginx

# Install a chart
helm install my-nginx bitnami/nginx

# Install with custom values
helm install my-nginx bitnami/nginx \
  --set replicaCount=3 \
  --set service.type=LoadBalancer

# Install with a values file
helm install my-nginx bitnami/nginx -f values.yaml

# List releases
helm list

# Upgrade a release
helm upgrade my-nginx bitnami/nginx --set replicaCount=5

# Rollback a release
helm rollback my-nginx 1

# Uninstall a release
helm uninstall my-nginx

# Show chart information
helm show chart bitnami/nginx
helm show values bitnami/nginx
```

Chart Structure

```
my-chart/
├─ Chart.yaml           # Chart metadata (name, version, description)
├─ values.yaml          # Default configuration values
├─ templates/           # Kubernetes manifest templates
│   ├─ deployment.yaml
│   ├─ service.yaml
│   ├─ ingress.yaml
│   └─ _helpers.tpl     # Template helper functions
├─ charts/              # Dependency charts
└─ README.md
```

When to Use Helm

- Deploying complex Kubernetes applications with many manifests
 - Need to manage different configurations for different environments
 - Using third-party applications (nginx, PostgreSQL, Redis, Prometheus)
 - Need rollback capabilities for K8s deployments
 - Sharing reusable application configurations across teams
-

6. IaC Tool Comparison

Feature	Terraform	CFT	Config Connector	Helm
Manages	Any cloud resource	GCP resources (Terraform modules)	GCP resources	K8s resources
Language	HCL	HCL (Terraform modules)	YAML	YAML + Go templates
Scope	Multi-cloud	GCP only	GCP only	Kubernetes only
Requires	Terraform CLI	Terraform CLI	GKE cluster	K8s cluster
State	State file	State file	K8s API	K8s API
Best for	General IaC, multi-cloud	Best-practice GCP setups	K8s-native GCP management	K8s app packaging

5. Cloud Deployment Manager

- Google Cloud's native IaC tool using YAML/Python/Jinja2 templates
- Exam expects familiarity with the concept and basic commands

Basic structure (config.yaml)

```
imports:
- path: vm_template.jinja

resources:
- name: my-vm
  type: compute.v1.instance
  properties:
    zone: us-central1-a
    machineType: zones/us-central1-a/machineTypes/n1-standard-1
    disks:
      - deviceName: boot
        type: PERSISTENT
        boot: true
        autoDelete: true
        initializeParams:
          sourceImage: projects/debian-cloud/global/images/family/debian-12
    networkInterfaces:
      - network: global/networks/default
```

gcloud commands

```
gcloud deployment-manager deployments create my-deployment --config config.yaml
gcloud deployment-manager deployments update my-deployment --config config.yaml
gcloud deployment-manager deployments list
gcloud deployment-manager deployments describe my-deployment
gcloud deployment-manager deployments delete my-deployment
```

- Deployment Manager uses a Google-managed SA to create resources; this SA needs appropriate IAM roles
- **Exam tip:** For ACE, know the concept and basic gcloud commands; Terraform is preferred in practice but Deployment Manager is GCP-native

6. Terraform Workspaces

- Manage multiple environments (dev, staging, prod) from the same Terraform code

```
terraform workspace new dev
terraform workspace new prod
terraform workspace list
terraform workspace select prod
terraform workspace show      # prints current workspace name
```

- Each workspace has its own state file in the backend
- Reference current workspace in config: `terraform.workspace`
- Example: `name = "my-app-${terraform.workspace}"`
- **Exam tip:** Workspaces are simpler than separate state files/backends for environment separation; use with caution for large infra (separate root modules preferred by Terraform best practices)

7. Kustomize

- Kubernetes-native configuration management (built into kubectl since 1.14)
- No templating engine — uses strategic merge patches and JSON patches
- Use cases: environment-specific overlays without duplicating YAML

Structure

```
base/
  deployment.yaml
  service.yaml
  kustomization.yaml
overlays/
  prod/
    kustomization.yaml  # patches for prod
  dev/
    kustomization.yaml  # patches for dev
```

base/kustomization.yaml

```
resources:
- deployment.yaml
- service.yaml
```

overlays/prod/kustomization.yaml

```
bases:
- ../../base
namePrefix: prod-
replicas:
- name: my-app
  count: 5
images:
- name: my-image
  newTag: "2.0.0"
```

Commands

```
kubectl apply -k overlays/prod/
kubectl diff -k overlays/prod/
kustomize build overlays/prod/ | kubectl apply -f -
```

- **Exam tip:** Kustomize is the Kubernetes-native alternative to Helm for configuration management without packaging

Exam Practice Questions

1. **Your team needs to deploy identical infrastructure across dev, staging, and production environments. Which approach should you use?**
 - Answer: **Terraform** with different variable files (`.tfvars`) for each environment. Use the same Terraform configuration with environment-specific values.
2. **Where should Terraform state be stored for a team of 5 engineers working on the same GCP project?**
 - Answer: In a **GCS backend** (`backend "gcs"`) with state locking enabled. This allows concurrent access while preventing conflicts.
3. **A team already uses ArgoCD and wants to manage GCP resources (Cloud SQL, Cloud Storage) using the same GitOps workflow. What should they use?**
 - Answer: **Config Connector** — Manages GCP resources as Kubernetes custom resources, integrating naturally with ArgoCD GitOps pipelines.
4. **You need to deploy a complex application (nginx + Redis + PostgreSQL) to GKE quickly using community-maintained configurations. What tool is most appropriate?**

- Answer: **Helm** — Install community charts from Bitnami or other repositories. Each component can be deployed with `helm install`.

5. What is the Cloud Foundation Toolkit (CFT)?

- Answer: A collection of **opinionated, production-ready Terraform modules** built by Google that implement GCP best practices for common architectures (projects, networks, GKE clusters, etc.).

6. You accidentally deleted a resource manually that Terraform manages. What happens on the next `terraform apply` ?

- Answer: Terraform detects the resource is missing from GCP (but still in state) and will **recreate it** to match the desired configuration.

Glossary

ACE (Associate Cloud Engineer) — A Google Cloud certification exam that validates skills in deploying, managing, and operating cloud solutions on GCP.

ArgoCD — An open-source GitOps continuous delivery tool for Kubernetes that declaratively manages application deployments from Git repositories.

Backend — In Terraform, the configuration that determines where the state file is stored and optionally where operations are executed (e.g., local disk, GCS, Terraform Cloud).

Base (Kustomize) — The set of Kubernetes manifests that serve as the foundational configuration, referenced by one or more overlays in a Kustomize directory structure.

CFT (Cloud Foundation Toolkit) — A collection of opinionated, production-ready Terraform modules built by Google that implement GCP best practices for common architectures.

Chart — In Helm, a packaged collection of Kubernetes resource template files and metadata that defines a complete application deployment.

CI/CD (Continuous Integration / Continuous Delivery) — A software development practice that automates building, testing, and deploying code changes; IaC configurations can be managed through CI/CD pipelines.

Cloud Deployment Manager — GCP's native infrastructure-as-code service that uses YAML, Python, or Jinja2 templates to define and deploy GCP resources.

Cloud SQL — GCP's fully managed relational database service supporting MySQL, PostgreSQL, and SQL Server; provisionable via CFT modules.

Cloud Storage — GCP's scalable object storage service; used as a Terraform remote backend (GCS) for storing state files.

Config Connector — A Kubernetes add-on that allows GCP resources to be managed using Kubernetes-style YAML manifests and kubectl, treating GCP resources as Kubernetes custom resources.

Configuration Drift — The divergence between the declared desired state of infrastructure and its actual running state, which IaC tools help detect and remediate.

Custom Resource — A Kubernetes API extension that allows users to define and manage new types of objects beyond the built-in Kubernetes resource types; Config Connector uses custom resources to represent GCP resources.

Data Source — In Terraform, a read-only reference to information about existing infrastructure resources that can be used in configuration without managing the resource.

Declarative — A configuration style that describes the desired end state of infrastructure (what should exist), in contrast to imperative scripts that describe how to achieve it step by step.

Dry Run — An operation that previews what changes would be made without actually applying them; `terraform plan` performs a dry run.

etcd — A distributed key-value store used by Kubernetes as its primary data store; Config Connector stores GCP resource state through the Kubernetes API backed by etcd.

Flux — An open-source GitOps tool for Kubernetes that automatically synchronizes cluster state with Git repositories.

GCS (Google Cloud Storage) — GCP's object storage service; used as a Terraform remote backend for team-shared state storage and locking.

GCP (Google Cloud Platform) — Google's suite of cloud computing services, the primary target platform for the tools covered in this chapter.

GKE (Google Kubernetes Engine) — GCP's managed Kubernetes service; required to run Config Connector and can be managed via CFT Terraform modules.

GitOps — An operational model that uses Git as the single source of truth for declarative infrastructure and application configuration, with automated reconciliation to the desired state.

Go Templates — The templating language used in Helm chart templates to generate Kubernetes manifests dynamically from values files.

HA (High Availability) — A design principle ensuring a system remains operational with minimal downtime; referenced in GCP architecture patterns managed with CFT.

HCL (HashiCorp Configuration Language) — The domain-specific declarative language used to write Terraform configuration files; also used for Cloud Foundation Toolkit modules.

Helm — A package manager for Kubernetes that bundles Kubernetes manifests into reusable, versioned packages called charts, with support for templating and release management.

Host Project — In GCP, the project that owns a Shared VPC network; referenced in CFT network modules.

IaC (Infrastructure as Code) — The practice of managing and provisioning cloud infrastructure through machine-readable configuration files rather than manual processes.

IAM (Identity and Access Management) — GCP's access control system; Deployment Manager uses a Google-managed service account with IAM roles to create resources.

ImagePullSecrets — Kubernetes objects that store credentials for authenticating to private container registries when pulling images.

Jinja2 — A Python-based templating engine supported by Cloud Deployment Manager for generating dynamic resource configurations.

JSON Patch — A format for describing changes to a JSON document; used by Kustomize as an alternative to strategic merge patches for modifying Kubernetes manifests.

K8s — Common abbreviation for Kubernetes.

kubectl — The command-line tool for interacting with Kubernetes clusters; used to apply Config Connector manifests and Kustomize configurations.

Kubernetes — An open-source container orchestration system for automating deployment, scaling, and management of containerized applications; the foundation for GKE, Config Connector, and Helm.

Kustomize — A Kubernetes-native configuration management tool built into kubectl that customizes raw YAML manifests using patches and overlays without templating.

Module — In Terraform, a reusable container of resources grouped together; CFT provides pre-built modules for common GCP services.

Multi-cloud — The practice of using services from multiple cloud providers; Terraform supports multi-cloud resource management through providers.

Nginx — A popular open-source web server and reverse proxy; commonly used as an example Helm chart deployment (bitnami/nginx).

Output — In Terraform, a named value exposed after `terraform apply` completes, such as an IP address or resource ID, used to share information between modules or with operators.

Overlay (Kustomize) — Environment-specific Kustomize configuration that patches a base configuration, allowing different settings for dev, staging, and production without duplicating YAML.

PostgreSQL — An open-source relational database; commonly deployed on GKE using Helm charts (e.g., bitnami/postgresql).

Provider — In Terraform, a plugin that manages the lifecycle of resources for a specific cloud platform or service (e.g., the `google` provider for GCP).

Redis — An open-source in-memory data store; commonly deployed on GKE using Helm charts.

Release — In Helm, a specific running instance of a chart deployed to a Kubernetes cluster, identified by a user-defined name.

Remote Backend — A Terraform backend configuration that stores state outside of the local filesystem, enabling team collaboration and state locking (e.g., GCS bucket).

Repository — In Helm, a collection of charts hosted at a URL that can be searched and installed from; in Terraform, a source location for modules.

Resource — In Terraform, a block that declares a piece of infrastructure to be created and managed (e.g., a VM, VPC, or firewall rule).

Root Module — The main Terraform configuration directory from which `terraform` commands are run; recommended over workspaces for large-scale environment separation.

SA (Service Account) — A GCP identity used by applications or services (rather than humans); Deployment Manager uses a Google-managed SA to provision resources.

State — In Terraform, a JSON file (`terraform.tfstate`) that records the current known state of all managed resources, used to calculate diffs and plan changes.

State File — The file (`terraform.tfstate`) or remote object that stores Terraform's record of managed resources; must be protected and never edited manually.

State Locking — A Terraform mechanism that prevents concurrent modifications to the state file by multiple users or pipelines; supported by GCS backends.

Strategic Merge Patch — A Kubernetes-specific patching strategy used by Kustomize to merge changes into existing manifest fields rather than replacing entire objects.

Terraform — An open-source infrastructure-as-code tool by HashiCorp that uses HCL to define, plan, and provision infrastructure across multiple cloud providers.

Terraform Cloud — HashiCorp's managed remote backend service for Terraform that provides state storage, locking, and enterprise features.

Terraform Workspace — A Terraform feature that maintains separate state files within the same backend for managing multiple environments (dev, staging, prod) from a single configuration.

tfvars — Terraform variable definition files (`.tfvars` extension) used to supply environment-specific values to input variables without modifying the main configuration.

Variable — In Terraform, an input parameter that allows configuration values to be passed in at runtime, enabling reusable and environment-agnostic configurations.

VPC (Virtual Private Cloud) — A logically isolated network on GCP; provisionable via the `terraform-google-network` CFT module.

YAML (YAML Ain't Markup Language) — A human-readable data serialization format used for Kubernetes manifests, Config Connector resources, Helm values files, Kustomize configurations,

and Cloud Deployment Manager templates.